1. Testing Apply For Book Feature

i. inorder to test the apply book feature, branch coverage is the most ideal method, because the code for that feature has conditions to be checked which lead to multiple options, thus there are multiple branches in the code, therefore it is ideal to use branch coverage. **comparing with path coverage method** which is comprehensive but can lead to a lot of possible combinations and makes testing complex with too many conditions. **statement coverage** only ensures that every line is executed, which might miss edge cases and decision outcomes. **linear independent path method** focus on unique paths but might not cover combinations of branching conditions adequately.

ii. The module has several conditional statements controlling its logic flow. Branch coverage ensures every decision point is tested for both true and false outcomes, making it a balanced choice.

iii. Usually white box testing is applied at unit level of testing because there modules are tested and comprehensive testing is required at this level. testing process should be stopped when maximum code coverage is achieved that is up to 80% code coverage. At this point testing can be stopped.

Testing the component

Code:

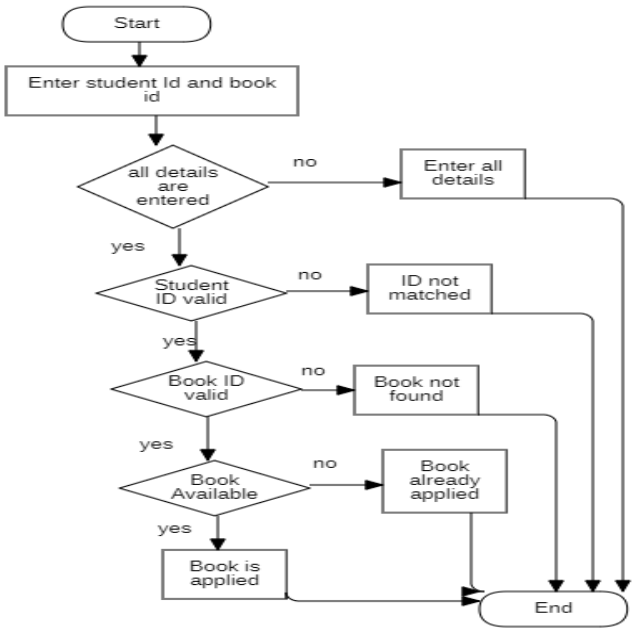
```
183 # Issuebook Button
184 def issuebooks():
185     global stdid
186     connectdb()
187 
188     qq="SELECT * FROM book WHERE serial = '%i'"
189     i=datetime.datetime(int(com1y.get()),month.index(com1m.get())+1,int(com1d.get()))
190     e=datetime.datetime(int(com2y.get()),month.index(com2m.get())+1,int(com2d.get()))
191     i=i.isoformat()
192     e=e.isoformat()
193 
194     if e1.get() == '' or e4.get() == '':
195             messagebox.showerror("ApplyBook", "Enter All Details")
196             return None
197 
198     cur.execute(qq%(int(e4.get())))
199     bkid = cur.fetchone()
200     qqq="SELECT userid FROM login WHERE userid = '%i'"
201     cur.execute(qqq%(int(e1.get())))
202     result = cur.fetchone()
203     usdi = (int(e1.get()))
204 
205     if usdi == stdid:
206         messagebox.showinfo("ApplyBook","ID Matched")
207         if bkid is None:
208             messagebox.showerror("ApplyBook","Book Not Found.")
209             closedb()
210         else:
211             qqqq='SELECT serial FROM bookissue where serial="%i"'
212             cur.execute(qqqq%(int(e4.get())))
213             chkk = cur.fetchone()
214             print(chkk)
215 
```

```
if usdi == stdid:
    messagebox.showinfo("ApplyBook","ID Matched")
    if bkid is None:
        messagebox.showerror("ApplyBook","Book Not Found.")
        closedb()
    else:
        qqqq='SELECT serial FROM bookissue where serial="%i"'
        cur.execute(qqqq%(int(e4.get())))
        chkk = cur.fetchone()
        print(chkk)

        if chkk is None:
            q='INSERT INTO BookIssue VALUE("%s","%s","%s","%s")'
            cur.execute(q%(e1.get(),(int(e4.get())), i, e))
            con.commit()
            win.destroy()
            messagebox.showinfo("ApplyBook", "Book Applied")
            closedb()
            libr()
        else:
            messagebox.showerror("ApplyBook", "Book Already Applied")

else:
    messagebox.showerror("ApplyBook","ID Not Matched")
```

FlowChart:



Test Cases:

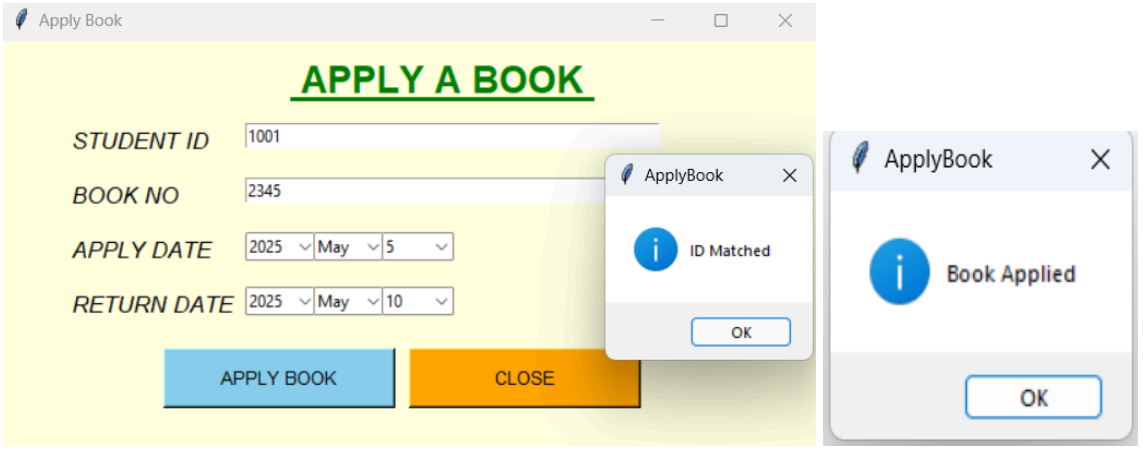
Project Name : Library Management System

Test Case Summary	
Test Case ID: LMS-01	Test designed by : Fatima
Test Priority: High	Test designed date : 12/1/2025
Module Name : Apply Books	Test executed date : 2/2/25
Test Title : Verifying Apply Books Functionality	Test executed by : Fatima
Description : testing apply book page	
Pre-condition: User enters valid information	
Dependencies: None	

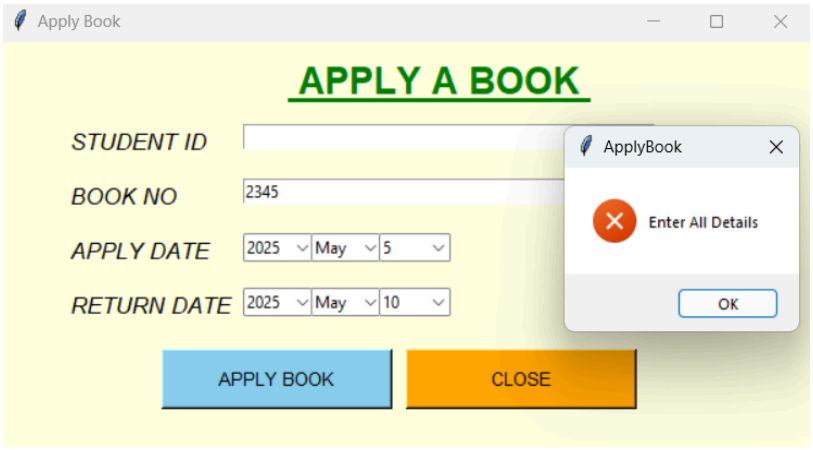
S.No	Test Case	Test Case Description	Test Data	Expected Output	Actaul Output	Status	Comment	Code Coverage
1	TC-001	Validate Apply Book Feature	student id: 1001 book id: 2345	Book Applied	Book Applied	pass	No Comments	4/8 = 50%
2	TC-002	validate null input	student id: null book id: null	Enter all details	Enter all details	pass	No Comments	1/8 =12.5%
3	TC-003	validate book is not available	student id: 1001 book id: 23422	book already applied	book already applied	pass	No Comments	1/8 = 12.5%
4	TC-004	validate invalid student id	student id: 1002 book id: 2346	ID not matched	ID not matched	pass	No Comments	1/8 = 12.5%
5	TC-005	validate invalid book id	student id: 1001 book id: 2347	Book not found	Book not found	pass	No Comments	1/8 = 12.5%

Output:

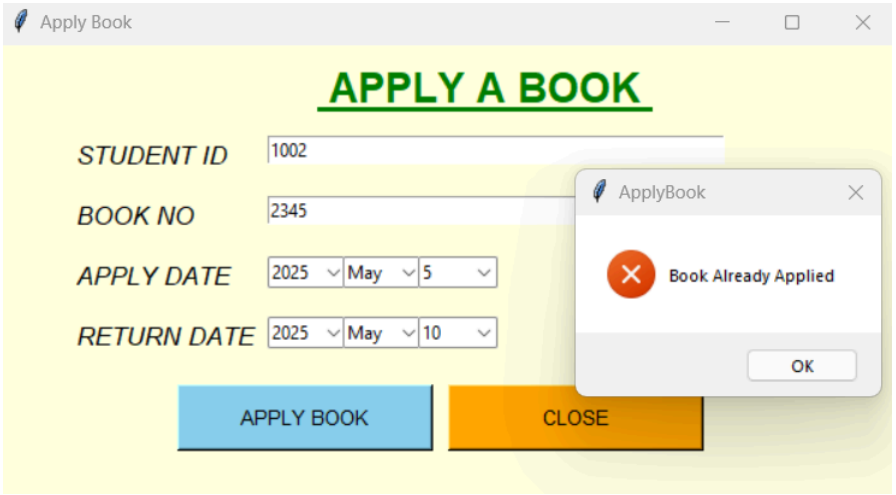
TC-001



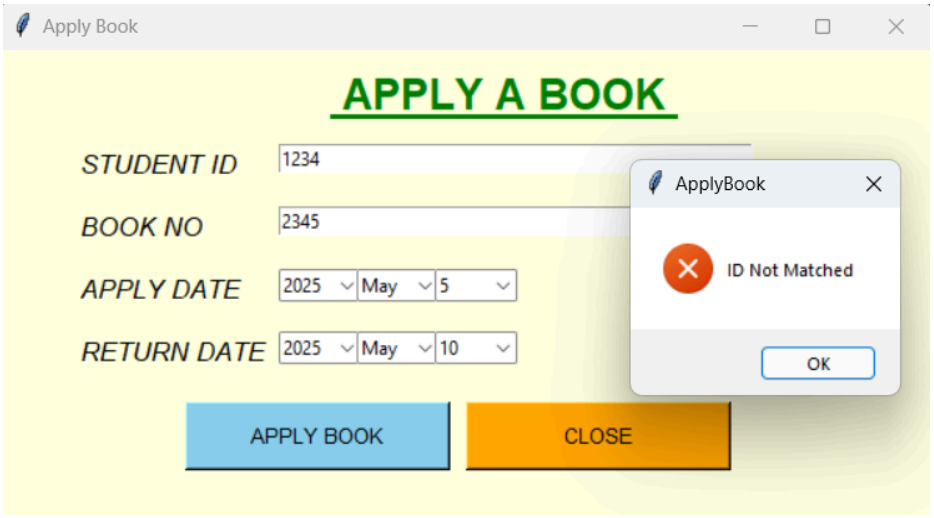
TC-002



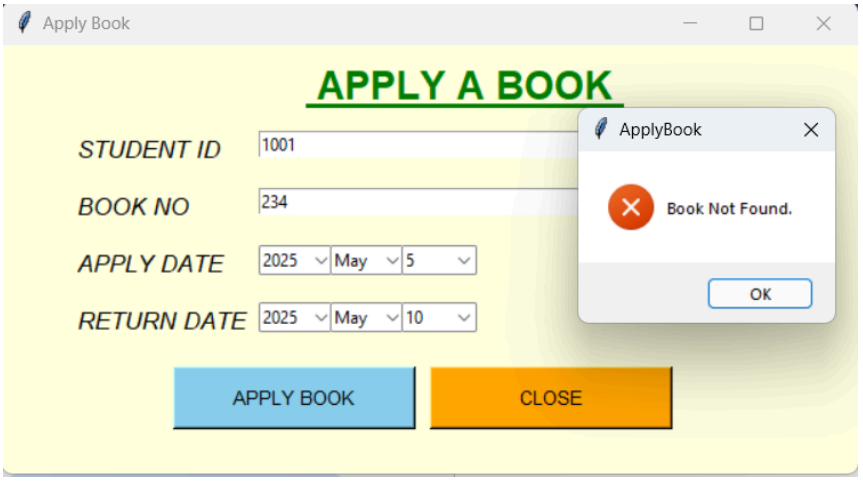
TC-003



TC-004



TC-005



2. Testing Add Student Feature

i. Compare statement coverage with other white box techniques

Statement coverage is a fundamental white box testing technique that ensures each line or statement in the source code is executed at least once during testing. It is especially effective for small, straightforward functions because it is simple to implement and quickly identifies any unused or unreachable code. However, unlike more advanced techniques like branch coverage, which checks all possible outcomes of conditional statements. Similarly, condition coverage digs deeper by testing each boolean sub-condition independently, and path coverage examines all possible execution paths through the code, making them more comprehensive but also more complex and harder to apply to larger or deeply nested code. While loop coverage focuses on testing loops under different iteration conditions statement coverage simply verifies that the loop is entered. Overall, for small-scale modules like addstudents(), statement coverage is often sufficient and more practical, while other techniques are better suited for more complex or critical components.

ii. Why you choose this technique?

Statement Coverage

because it is simple, efficient, and sufficient for the small addstudents function. Covers all execution lines for primary functional assurance. The function is short and mostly linear with a few conditions. Statement coverage allows confirming that each line of logic is executed at least once.

Iii. On what level we can apply white box and why?

White-box testing is best applied at the **unit and integration testing levels** because it requires access to internal code logic, while it is **not suitable for system or acceptance testing** which focus on external behavior without code visibility.

CODE:

```
def addstudents():
    global con,cur
    connectdb()
    q="SELECT * FROM login WHERE userid = '%i'"

    if e1.get() == '' or e2.get() == '' or e3.get() == '' or e4.get() == '' or e5.get() == '':
        messagebox.showerror("Student", "Fill All Details")
        return None

    cur.execute(q%(int(e2.get())))
    student = cur.fetchone()

    if student is None:
        qq='INSERT INTO Login VALUE("%s","%i","%i","%s","%i")'
        try:
            pas = int(e3.get())
        except ValueError:
            messagebox.showerror("Student", "Password Must be in Numbers")
            return None
        cur.execute(qq%(e1.get(),int(e2.get()),int(e3.get()),e4.get(),int(e5.get())))
        con.commit()
        win.destroy()
        messagebox.showinfo("Student", "Student Added")
        closedb()
        admin()
    else:
        messagebox.showerror("Student","ID Already Taken.")
        closedb()
```

For Code coverage:

1. connectdb() # Line 1
2. q = "SELECT..." # Line 2
3. if e1.get() == '' or ... # Line 3
4. messagebox.showerror(...) # Line 4 (inside if)
5. return None # Line 5 (inside if)
6. cur.execute(...) # Line 6
7. student = cur.fetchone() # Line 7
8. if student is None: # Line 8
9. qq = 'INSERT INTO ...' # Line 9 (inside if)
10. try: pas = int(...) # Line 10 (inside if)
11. except ValueError: ... # Line 11 (inside if)
12. cur.execute(...) # Line 12 (inside try)
13. con.commit() # Line 13
14. win.destroy() # Line 14
15. messagebox.showinfo(...) # Line 15

16. closedb() # Line 16
17. admin() # Line 17
18. else: # Line 18
19. messagebox.showerror(...) # Line 19 (inside else)
20. closedb() # Line 20

Test Summary

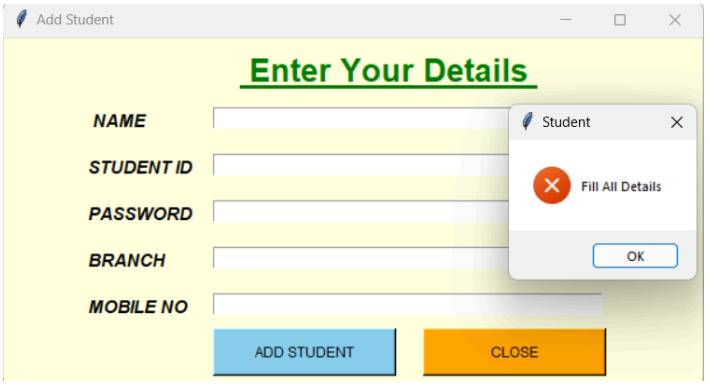
Project Name	Library Management System
Test Suite ID:	LMS-02
Test Title:	Add Student
Test Priority:	High
Module Name:	Add Student
Designed By:	Fathima Raihaan
Designed Date:	21/04/2025
Executed By:	Fathima Raihaan
Executed Date:	22/04/2025
Description of Test:	Testing the add students feature
PREREQUISITES:	
Pre-Condition:	Logged in as admin
Dependencies:	

S.no	**Test Case**	**Test Steps**	**Test Data**	**Expected Output**	**Actual Output**	**Pass/ Fail**	**Comments ?**	**Code Coverage**
TC-01	All fields empty	1. Enter details 2. Click Add student	"", "", "", "", ""	Error: “Fill All Details”	Fill All Details	PASS		5/20=25%
TC-02	All valid data	1. Enter details 2. Click	"Zara", "1002", "1234",	Student Added	Student Added	PASS		15/20 = 75%

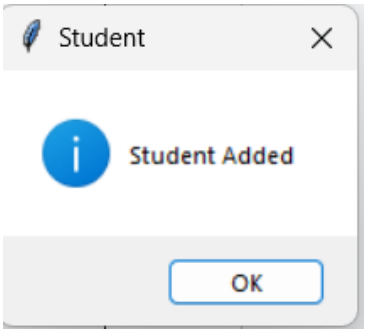
		Add student	"CS", "999999 9999"					
TC-03	Valid fields, non-numeric password	1. Enter details 2. Click Add student	"Sara", "1004", "abcd", "IT", "888888 8888"	Error: “Password Must be in Numbers.”	Password Must be in Numbers	PASS		11/20 = 55%
TC-04	Student ID already exists in DB	1. Enter details 2. Click Add student	"Zaid", "1002", "2345", "ME", "777777 7777"	Error: “ID Already Taken.”	ID Already Taken	PASS		10/20 = 50%
TC-05	Invalid ID input (non-numeric)	1. Enter details 2. Click Add student	"Alex", "abc", "1234", "CS", "999999 9999"	TypeError	Add student button doesn’t get clicked	FAIL	An output should be displayed with the error as “enter valid ID”	6/20 = 30%
TC-06	The mobile number is invalid	1. Enter details 2. Click Add student	"John", "1006", "4567", "CS", "99abc9 999"	TypeError	Add student button doesn’t get clicked	FAIL	An output should be displayed with the error as “enter valid mobile number”	8/20=40%

ACTUAL OUTPUT

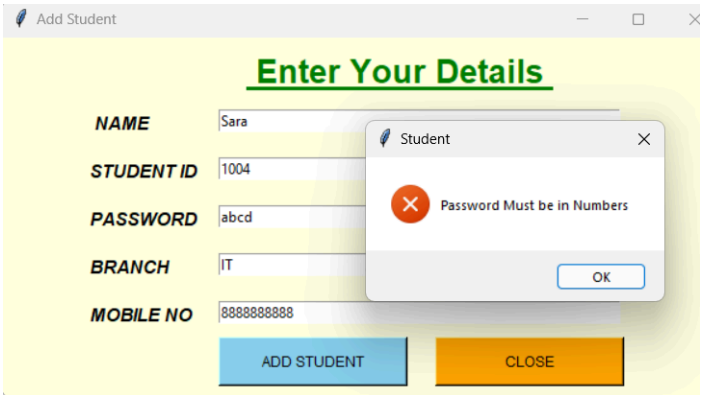
TC-01



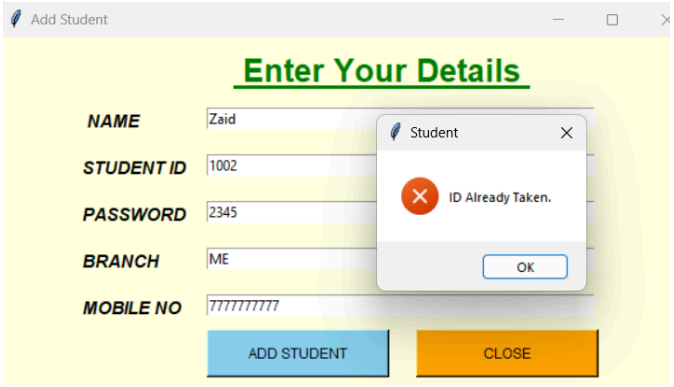
TC-02



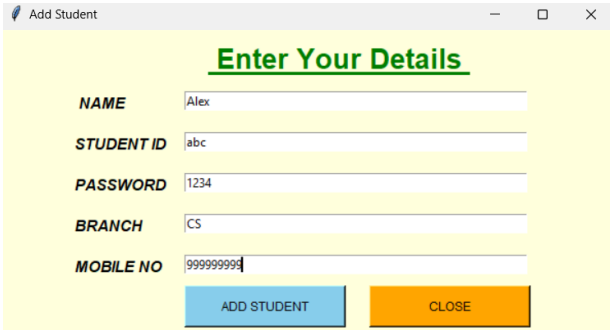
TC-03



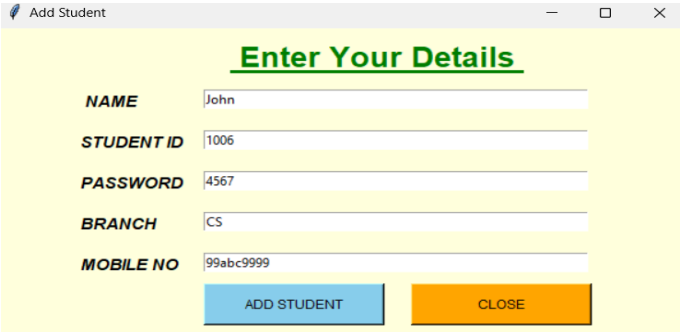
TC-04



TC-05



TC-06



3. Testing Delete Student Feature

i. Compare Branch coverage with other white box techniques

Branch coverage ensures all possible True/False outcomes of decisions are tested. It is more thorough than statement coverage, which only checks line execution. Condition coverage tests individual conditions but not full outcomes. Path coverage is more comprehensive but complex. Branch coverage offers a good balance of depth and simplicity for functions with multiple conditions.

ii. Why You Chose This Technique?

Branch coverage is efficient and sufficient for the deleteStudent()function. It ensures all decision points are tested (e.g., password check, empty ID) and provides better assurance than statement coverage for conditional logic

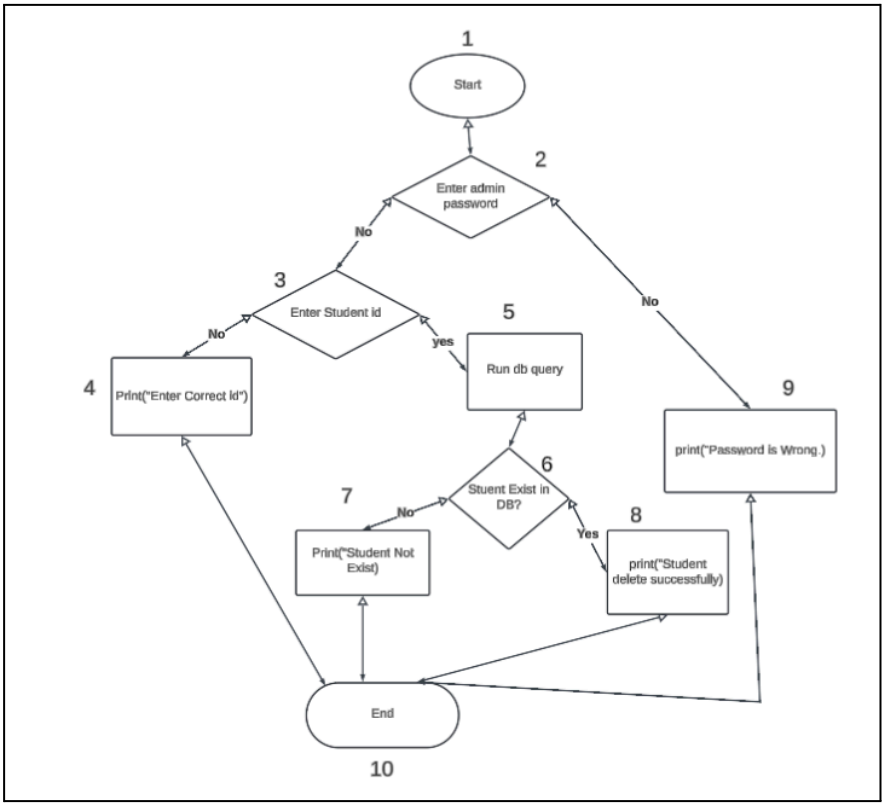
.iii. On What Level Can We Apply White Box and Why?

White-box testing, including branch coverage, is best at the unit and integration levels where internal logic is visible. It's not suitable for system or acceptance testing, which focus on external behavior.

Code:

```
680  # Deletestudents query
681  def deletestudents():
682      connectdb()
683      if e2.get()=='admin':
684          q="SELECT * FROM login WHERE userid = '%i'"
685
686          if e1.get() == '':
687              messagebox.showerror("Delete", "Enter Correct ID")
688              return None
689
690          cur.execute(q%(int(e1.get())))
691          book = cur.fetchone()
692
693          if book is None:
694              messagebox.showerror("Delete","Student Not Found.")
695              closedb()
696          else:
697              qq="DELETE FROM login WHERE userid = '%i'"
698              cur.execute(qq%(int(e1.get())))
699              con.commit()
700              win.destroy()
701              messagebox.showinfo("Delete","Student Deleted Successfully.")
702              closedb()
703              admin()
704
705      else:
706          messagebox.showerror("Delete","Password is Wrong.")
707          closedb()
708
```

Flow Chart



Test Summary :

Project Name:	**Delete Student**
Test Suite ID:	DeleteStudent_TESE61
Test Title:	Test the Delete Student
Test Priority:	Medium
Module Name:	Delete Student
Designed By:	Urwa Qadir
Designed Date:	4/30/2025
Executed By:	Urwa Qadir
Executed Date:	4/30/2025
Description of Test:	To check the behavior of the deletestudents() function under different inputs
pre-condition	Login as a Admin

Test Case:

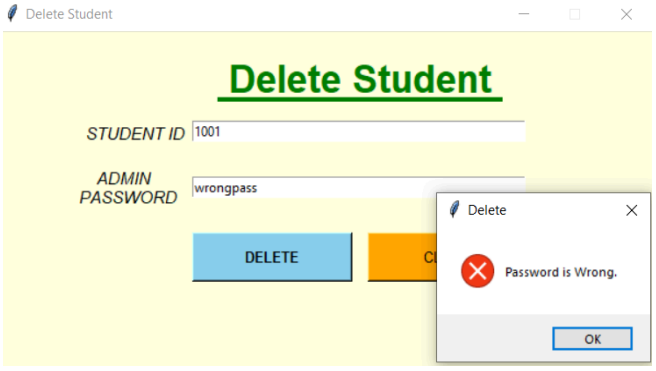
SNO	Test Case	Test Steps	Test Data	Expected Output	Actual Output (example)	Pass/Fail
1	TC1	1. Enter Student ID as '1001' 2. Enter password as 'wrongpass' 3. Click DELETE	e1 = '1001' e2 = 'wrongpass'	Show "Password is Wrong."	"Password is Wrong."	Pass
2	TC2	1. Leave Student ID empty 2. Enter password as 'admin' 3. Click DELETE	e1 = '' e2 = 'admin'	Show "Enter Correct ID"	"Enter Correct ID"	Pass
3	TC3	1. Enter non-existent ID (e.g. '9999') 2. Enter password as 'admin' 3. Click DELETE	e1 = '9999' e2 = 'admin'	Show "Student Not Found."	"Student Not Found."	Pass
4	TC4	1. Enter existing ID (e.g. '1001') 2. Enter password as 'admin' 3. Click DELETE	e1 = '1001' e2 = 'admin'	Show "Student Deleted Successfully."	"Student Deleted Successfully."	Pass

Output:

TC_0:1

Branch covered: 1-2-9-10

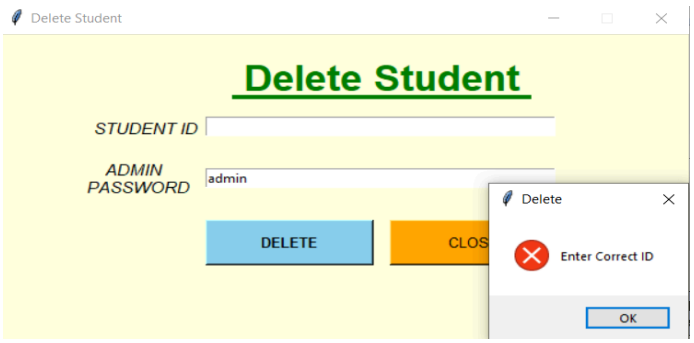
Branch Coverage=Total Branches/Branches Covered×100=1/6 x100=16.66%



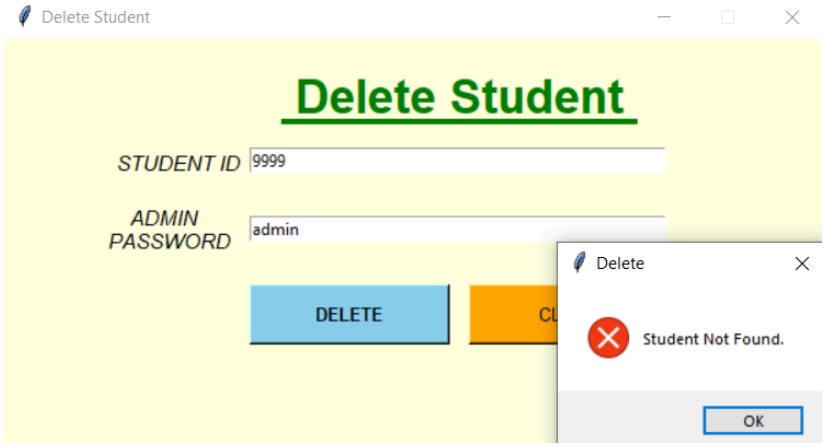
TC_02

Branch covered: 1-2-3-10

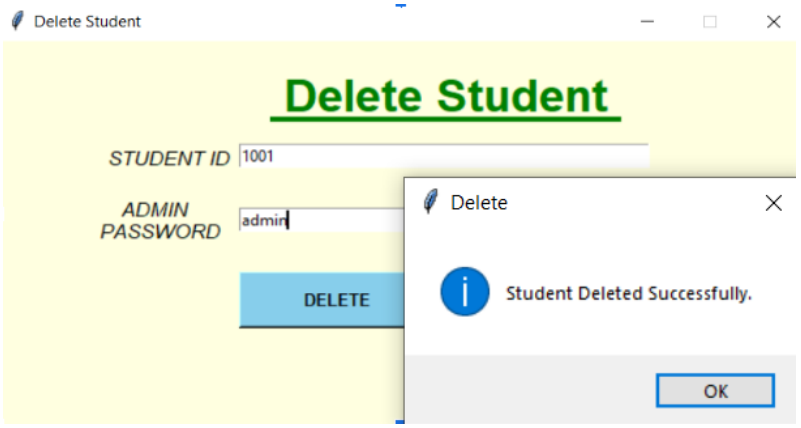
Branch Coverage=Total Branches/Branches Covered×100=2/6 x100=33.33%



TC_0:3
Branch covered: 1-2-3-5-6-7-10
Branch Coverage=Total Branches/Branches Covered×100=3/6 x 100=50%



TC_04:
Branch covered: 1-2-3-5-6-8-10
Branch Coverage=Total Branches/Branches Covered×100=3/6 x100=50%



4. TESTING ADD BOOK FEATURE

i. Comparison with Other White Box Techniques

I preferred **branch coverage** because it checks all decision paths (true/false), which fits our feature's logic checks. Statement coverage is too simple, and path coverage is too complex. Condition coverage is more detailed but not necessary here. Loop coverage isn't useful since we don't use loops. So, branch coverage was the most practical choice.

ii. Why Branch Coverage?

- Covers the all major decision points (field validation, serial checks).
- Ensures all logical paths are tested without being overly complex.
- It balances depth and practicality.

iii. Application Level & When to Stop Testing

Level: Applied at unit and integration level (function or module).

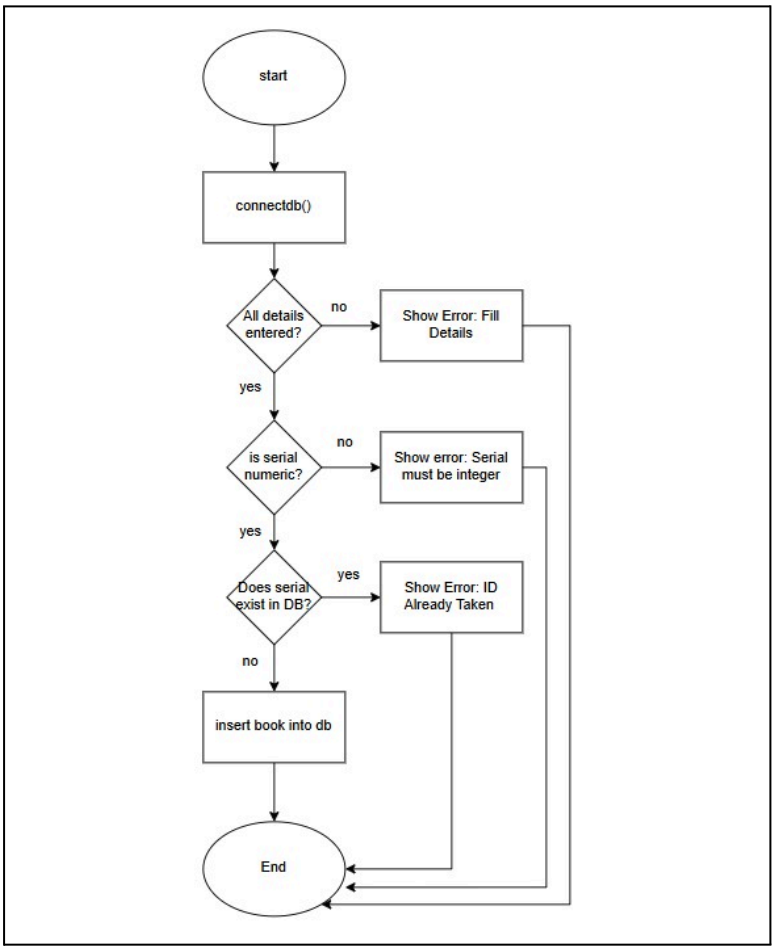
Stop testing when:

- All critical branches are covered (e.g., 100% branch coverage).
- No new bugs are found in successive tests.
- Time/budget constraints are met.
- Risk assessment deems the system stable.

CODE:

```
def addbooks():
    connectdb()
    global cur, con
    if not all([e1.get(), e2.get(), e3.get(), e4.get()]):
        messagebox.showerror("Book", "Fill All Details")
        return
    try:
        serial = int(e4.get())
    except ValueError:
        messagebox.showerror("Book", "Serial must be a number.")
        return
    cur.execute("SELECT * FROM book WHERE serial = ?", (serial,))
    iddd = cur.fetchone()
    if iddd is None:
        try:
            cur.execute("INSERT INTO Book VALUES (?, ?, ?, ?)", (e1.get(), e2.get(), e3.get(), serial))
            con.commit()
            win.destroy()
            messagebox.showinfo("Book", "Book Added")
        except Exception as e:
            messagebox.showerror("Book", f"Failed to add book: {e}")
        finally:
            closedb()
            libr()
    else:
        messagebox.showerror("Book", "ID Already Taken.")
        closedb()
```

FLOWCHART:



TEST SUMMARY:

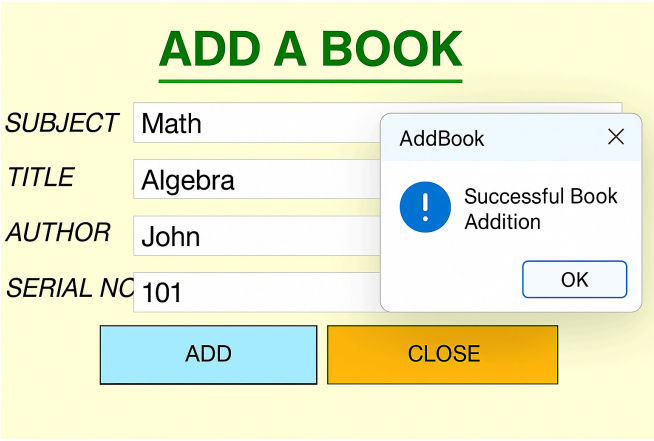
Project Name	Library Management System
Test Suite ID	AddBook-TESE60
Test Title	Add Book
Test Priority	High
Module Name	Add Book
Designed By	Namra Imtiaz
Designed Date	30/04/2025
Executed By	Namra Imtiaz
Executed Date	30//04/2025
Description of Test	Testing the add books feature

TEST CASES:

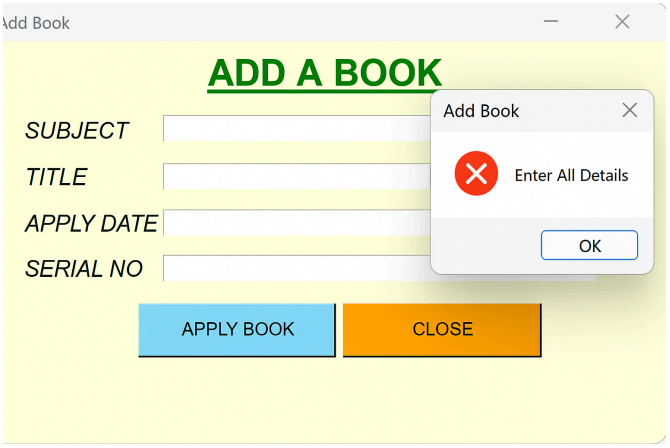
S.No	Test Case	Description	Test Data	Expected Output	Actual Output	Status	Comment	Code Coverage %
1	TC_01	Valid data, serial not in DB	["Math", "Algebra", "John", "105"]	Book Added	As expected	Pass	Tests successful book addition	4/4 = 100%
2	TC_02	All fields empty	["", "", "", ""]	Show error: Fill All Details	As expected	Pass	Tests branch: Fields not filled	1/4 = 25%
3	TC_03	Some fields empty	["Math", "", "John", "101"]	Show error: Fill All Details	As expected	Pass	Ensures validation triggers if any field is empty	1/4 = 25%
4	TC_04	Non-numeric serial	["Math", "Algebra", "John", "ABC"]	Show error: Serial must be integer	As expected	Pass	Tests branch: Invalid serial type	2/4 = 50%
5	TC_05	Serial already in DB	["Math", "Algebra", "John", "101"]	Show error: ID Already Taken	As expected	Pass	Tests: serial exists in DB	3/4 = 75%

OUTPUTS:

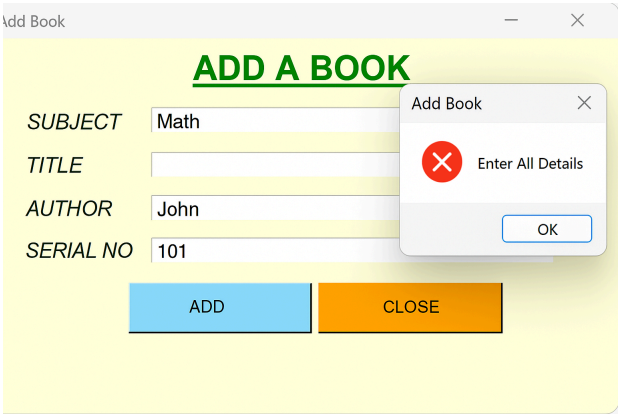
TC_01:



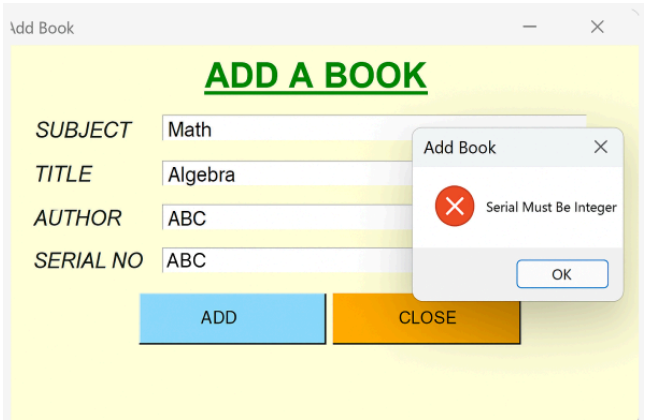
TC_02:



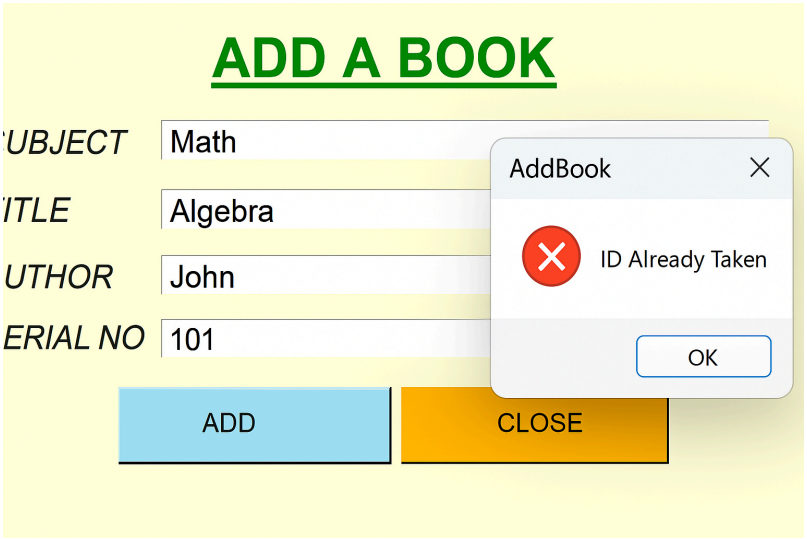
TC_03:



TC_04:



TC_05:



5. Testing Delete Books Feature

i. Comparison of White Box Testing Techniques

- Statement Coverage:
  Ensures every line of code is executed at least once. It's simple but does not check conditions or logical decisions, so some issues might be missed.
- Branch Coverage:
  Ensures every decision (like if statements) is tested for both true and false outcomes. It's more thorough than statement coverage but still doesn't cover all paths.
- Independent Path Coverage:
  Tests every unique path the code can take. This is the most thorough method but can be time-consuming and complex.

ii. Why I Chose Statement Coverage

I chose Statement Coverage because it is the simplest and quickest way to test that every line of code in the deletebooks() function is executed. It ensures that basic functionality works and provides a foundation for more detailed testing.

iii. When and Where to Apply White Box Testing, and When to Stop Testing

- When to Apply:
  White-box testing is used during unit testing and integration testing. It's applied to test individual functions and how different parts of the system interact.
- When to Stop:
  Testing should stop when all critical paths are covered, no major bugs are found, and the code reaches an acceptable level of coverage (usually 80-90%).

CODE:

```
def deletebooks():
    connectdb() #1
    if e2.get()=='admin': #2
        q="SELECT * FROM book WHERE serial = '%i'" #3
        if e1.get() == '':   #4
            messagebox.showerror("Delete", "Enter Correct ID") #5
            return None       #6
        cur.execute(q%(int(e1.get()))) #7
        book = cur.fetchone() #8
        if book is None: #9
            messagebox.showerror("Delete","Book Not Found.") #10
            closedb() #11
        else:    #12
            qq="DELETE FROM book WHERE serial = '%i'" #13
            cur.execute(qq%(int(e1.get()))) #14
            con.commit() #15
            win.destroy() #16
            messagebox.showinfo("Delete","Book Deleted Successfully.") #17
            closedb() #18
            admin()         #19
    else: #20
        messagebox.showerror("Delete","Password is Wrong.") #21
        closedb() #22
```

def deletebooks():

connectdb() #1

if e2.get() == 'admin': #2

q = "SELECT * FROM book WHERE serial = '%i'"

if e1.get() == '': #4

messagebox.showerror("Delete", "Enter Correct ID") #5

return None

cur.execute(q % (int(e1.get()))) #6

book = cur.fetchone() #7

if book is None: #8

messagebox.showerror("Delete", "Book Not Found.") #9

closedb() #10

else: #11

qq = "DELETE FROM book WHERE serial = '%i'"

cur.execute(qq % (int(e1.get()))) #12

con.commit() #13

win.destroy() #14

messagebox.showinfo("Delete", "Book Deleted Successfully.") #15

```
            closedb() #16
            admin() #17
    else: #18
        messagebox.showerror("Delete", "Password is Wrong.") #19
        closedb() #20
```

Summary Report

Project Name	Library Management System
Test Suite ID:	LMS-05
Test Title:	Delete Books
Test Priority:	Medium
Module Name:	Delete Books
Designed By:	Jassica Allen
Designed Date:	30/04/2025
Executed By:	Jassica Allen
Executed Date:	30/04/2025
Description of Test:	Testing the delete books feature
PREREQUISITES:	
Pre-Condition:	Logged in as admin
Dependencies:	

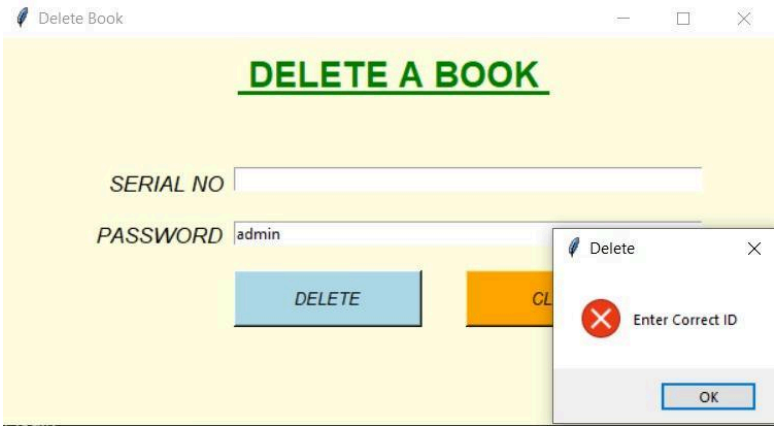
Test Case:

Test Case	Description	Test Data	Expected Output	Actual Output	Status	Covered Lines	Code Coverage %
TC1	Test for empty serial number	Password = "admin", Serial = ""	Error message: "Enter Correct ID"	Error message: "Enter Correct ID"	Pass	Lines 1, 2, 4, 5	(4/20) 20%
TC2	Test for incorrect password	Password = "wrong", Serial = "1"	Error message: "Password is Wrong"	Error message: "Password is Wrong"	Pass	Lines 1, 2, 18, 19, 20	(5/20)25%
TC3	Test for non-existent book	Password = "admin", Serial = "999"	Error message: "Book Not Found"	Error message: "Book Not Found"	Pass	Lines 1, 2, 4, 6, 7, 8, 9, 10	(8/20) 40%
TC4	Test for book deletion	Password = "admin", Serial = "123"	Success message: "Book Deleted Successfully"	Success message: "Book Deleted Successfully"	Pass	Lines 1, 2, 4, 6, 7, 8, 11, 12, 13, 14, 15, 16, 17	(13/20) 65%
TC5	Test for password correct, non-existent book	Password = "admin", Serial = "0"	Error message: "Book Not Found"	Error message: "Book Not Found"	Pass	Lines 1, 2, 4, 6, 7, 8, 9, 10, 18, 19, 20	(11/20) 55%

The above test cases shows statement coverage of 100% as each statement is executed and validated.

Output:

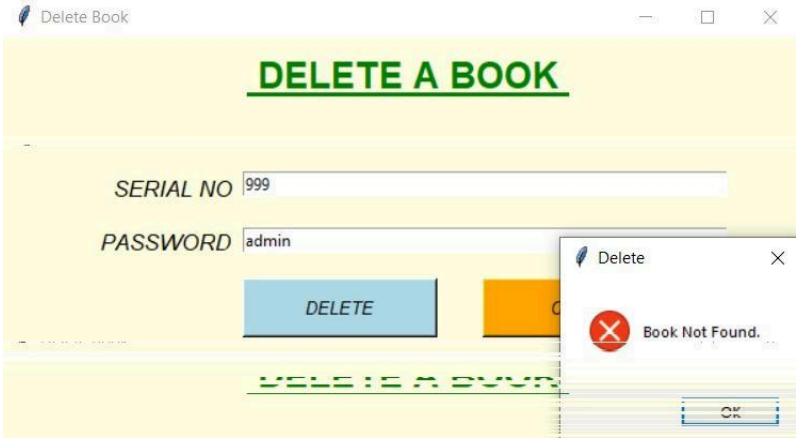
TC_1:


Delete Book
DELETE A BOOK
SERIAL NO
PASSWORD
admin
DELETE
Delete
Enter Correct ID
OK

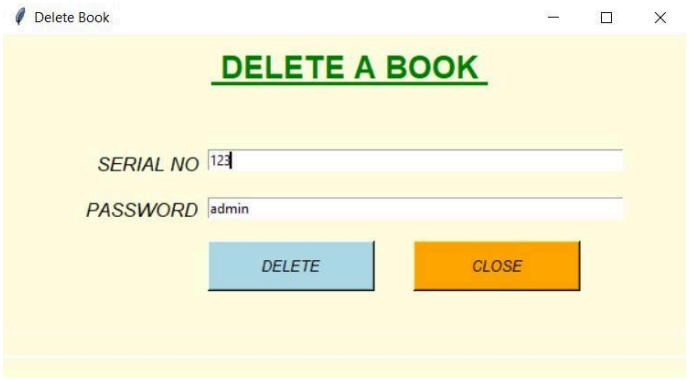
TC_2:

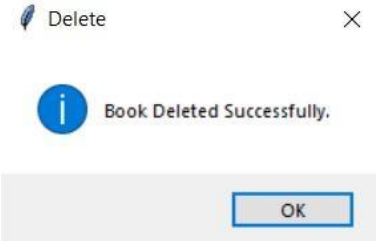

Delete Book
DELETE A BOOK
SERIAL NO
1
PASSWORD
wrong
DELETE
Delete
Password is Wrong.
OK

TC_3:

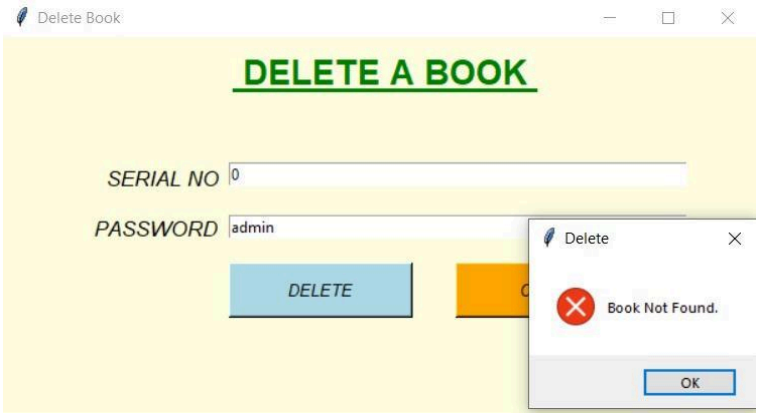

Delete Book
DELETE A BOOK
SERIAL NO
999
PASSWORD
admin
DELETE
Delete
Book Not Found.

TC_4:


Delete Book
DELETE A BOOK
SERIAL NO
123
PASSWORD
admin
DELETE
CLOSE


Delete
Book Deleted Successfully.
OK

TC_5:


Delete Book
DELETE A BOOK
SERIAL NO
0
PASSWORD
admin
DELETE
Delete
Book Not Found.
OK

Exercise: Select your previous project and Design Test cases for at least 5 features using any of black box strategy [you can choose different techniques to test different features] also answer the following questions

i. Draw a comparison of selected techniques with all other ones.

ii. Why you choose this technique?

iii. On what level we can apply black box and why?

Attached all printouts and screenshots of application [UI+ application o/p on test data] here. Also attached summary report.

1. Testing Apply Book Feature

i. Decision Table Testing was chosen over techniques like Boundary Value Analysis (BVA), Equivalence Class Partitioning (ECP) because it excels in handling complex systems with interdependent conditions. Unlike BVA and ECP, which focus on numeric ranges or input partitions, Decision Table Testing ensures comprehensive coverage of all input combinations and outcomes.

ii. Decision Table Testing is ideal for this application because it systematically handles the interdependent conditions of Student ID, Book No, and Apply/Return Dates. It ensures all possible input combinations are tested, which is crucial for detecting missing logic or unhandled scenarios in a multi-condition system. Additionally, it provides clarity in test case creation, ensuring logical completeness while avoiding redundancy, making it the most practical and reliable approach.

iii. Black box testing is usually applied at system level because it can ensure the overall system performance, focuses on validating that the application meets its functional requirements, such as verifying input validations and error handling, making it well-suited for end-to-end functionality testing in this case.

Testing the Feature

step 1: identify the conditions

Student ID Validity: Valid or Invalid.
Book No Validity: Valid or Invalid.
Apply Date Validity: Valid or Invalid.
Return Date Validity: Valid or Invalid (Return > Apply).

step 2: Identify possible actions

Invalid Student ID

Book Not Found.
Successfully apply the book.

step 3: Initial Decision Table

no of columns = 2^4 = 16

Conditions	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10	R11	R12	R13	R14	R15	R16
Student ID is Valid	T	T	T	T	T	T	T	T	F	F	F	F	F	F	F	F
Book No is Valid	T	T	T	T	F	F	F	F	T	T	T	T	F	F	F	F
Apply Date Valid	T	T	F	F	T	T	F	F	T	T	F	F	T	T	F	F
Return Date is Valid	T	F	T	F	T	F	T	F	T	F	T	F	T	F	T	F
Actions																
Book is Applied	✔															
Book is not applied		✔	✔	✔	✔	✔	✔	✔	✔	✔	✔	✔	✔	✔	✔	✔

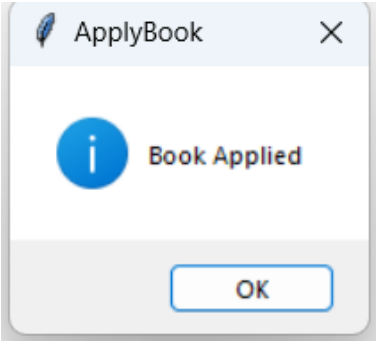
step 4: Reduced Decision Table

Conditions	R1	R2	R3-R4	R5-R8	R9-R16
Student ID is Valid	T	T	T	T	F
Book ID is Valid	T	T	T	F	╳
Apply Date is Valid	T	T	F	╳	╳
Return Date is Valid	T	F	╳	╳	╳
Actions					
Book is Applied	✔				
Book is not Applied		✔	✔	✔	✔

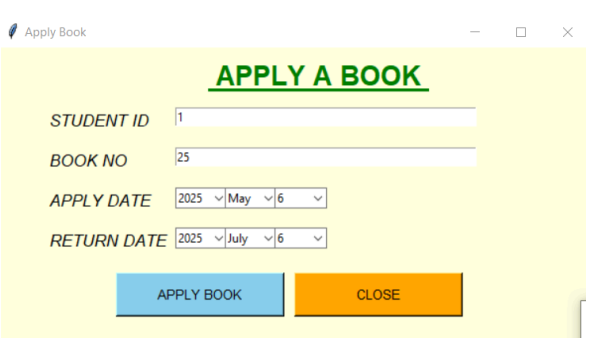
S.No	Test Case	Expected Output	Actaul Output	Status	Comment
TC-001	student id: valid book id: valid apply date: valid return	Book Applied	Book Applied	pass	No Comments
TC-002	student id: valid book id: valid apply date: valid return date: not valid	Enter all details	Book Applied	Fail	No Comments
TC-003	student id: valid book id: valid apply date: not valid	Enter all details	Enter all details	pass	No Comments
TC-004	student id: valid book id: not valid apply date: not valid return	Book not found	Book not found	pass	No Comments
TC-005	student id: not valid book id: not valid apply date: not valid return date: not	ID not matched	ID not matched	pass	No Comments

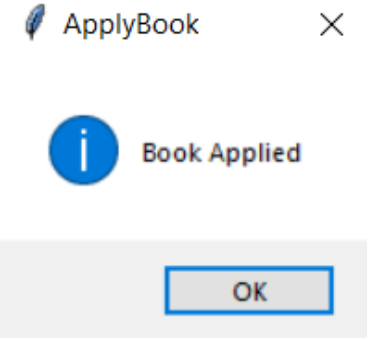
Outputs:

TC-001

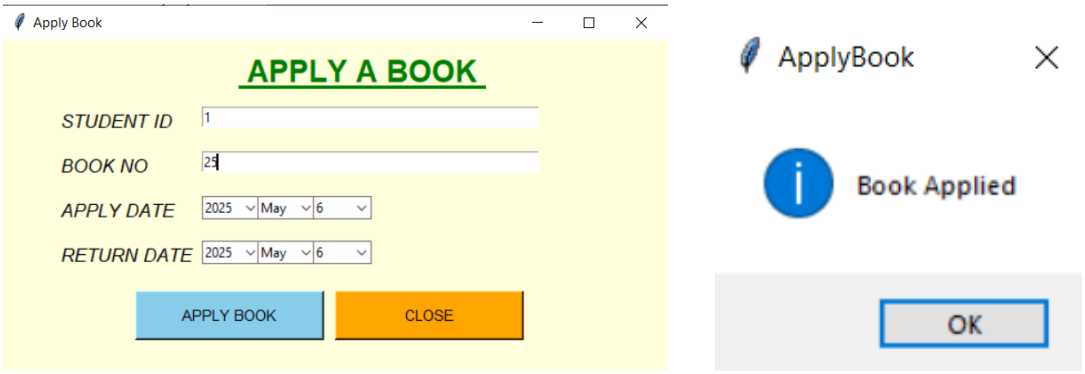


TC-002

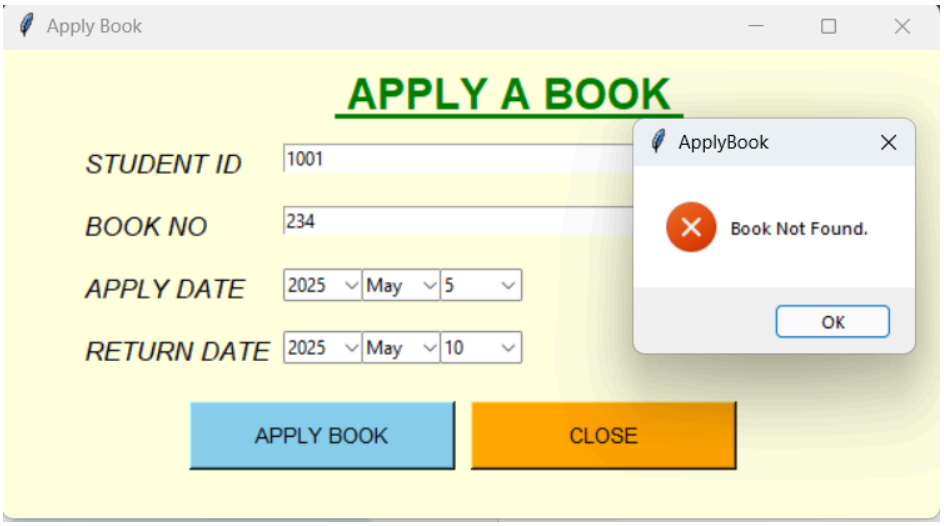




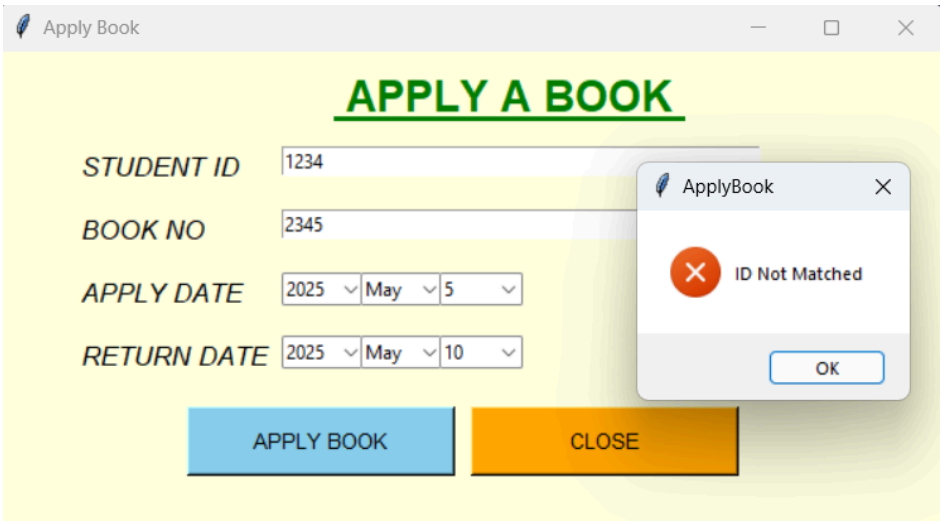
TC-003


Apply Book
APPLY A BOOK
STUDENT ID
1
BOOK NO
25
APPLY DATE
2025
May
6
RETURN DATE
2025
May
6
APPLY BOOK
CLOSE
ApplyBook
Book Applied
OK

TC-004


Apply Book
APPLY A BOOK
STUDENT ID
1001
BOOK NO
234
APPLY DATE
2025
May
5
RETURN DATE
2025
May
10
APPLY BOOK
CLOSE
ApplyBook
Book Not Found.
OK

TC-005


Apply Book
APPLY A BOOK
STUDENT ID
1234
BOOK NO
2345
APPLY DATE
2025
May
5
RETURN DATE
2025
May
10
APPLY BOOK
CLOSE
ApplyBook
ID Not Matched
OK

2. Testing Add Student Feature

i. Draw a comparison of selected techniques with all other ones.

Equivalence Class Partitioning (ECP) divides input data into valid and invalid partitions, allowing testers to select representative values instead of testing every possible input. Compared to other black box techniques, Boundary Value Analysis (BVA) focuses on values at the edges of these partitions, often catching more off-by-one errors. Decision Table Testing handles complex business logic by mapping inputs to expected outcomes, while State Transition Testing is ideal for systems with different states and transitions (e.g., login workflows). ECP is simpler and efficient for broad input coverage, but less precise for edge cases or logical rule combinations compared to these other methods.

ii. Why you choose this technique?

The "Add Student" form contains basic input fields: name, student ID, password, mobile number, and branch.

These fields are best suited for validation through ECP:

Name: valid/invalid strings

Student ID: valid/invalid numbers

Mobile No: valid/invalid formats

ECP helps reduce the number of test cases by grouping inputs, which is practical and efficient during form validation testing. It's also simple and quick to apply, especially in early testing phases.

iii. On what level we can apply black box and why?

System Testing and Acceptance Testing, because it treats the whole application as a black box; checks full behavior from user perspective,

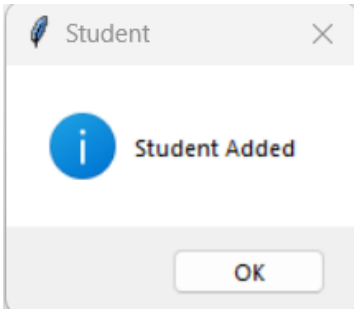
Project Name	Library Management System
Test Suite ID:	TSB-001
Test Title:	Add Student
Test Priority:	High
Module Name:	Add Student
Designed By:	Fathima Raihaan
Designed Date:	21/04/2025
Executed By:	Fathima Raihaan
Executed Date:	22/04/2025
Description of Test:	Testing the add students feature
PREREQUISITES:	
Pre-Condition:	Logged in as admin

Dependencies:	

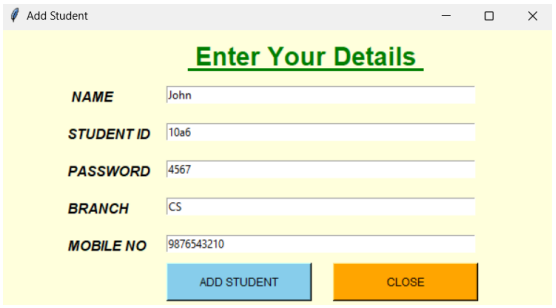
Test Case ID	Test Case Description	Test Data					Expected Result	Actual Result	Status
		Name	Student Id	Password	Branch	Mobile No			
TC-01	Valid data	John	1006	4567	CS	9876543210	Student Added	Student Added	PASS
TC-02	Invalid name	John123	1006	4567	CS	9876543210	Invalid Name	Student Added	FAIL
TC-03	Invalid ID	John	10a6	4567	CS	9876543210	Invalid Id	Add student button doesn't get clicked	FAIL
TC-04	Invalid Password	John	1006	abcd	CS	9876543210	Password Must be in Numbers	Password Must be in Numbers	PASS
TC-05	Invalid Branch	John	1006	4567	10	9876543210	Invalid Branch	Student Added	FAIL
TC-06	Invalid Mobile No.	John	1006	4567	CS	987ABC3210	Invalid Mobile No.	Add student button doesn't get clicked	FAIL
TC-07	Invalid Name	blank	1006	4567	CS	9876543210	Fill All Details	Fill All Details	PASS
TC-08	Invalid ID (upper bound)	John	99999	4567	CS	9876543210	Invalid ID	Student Added	FAIL
TC-09	Invalid Password (lower bound)	John	1006	12	CS	9876543210	Invalid Password	Student Added	FAIL

Output

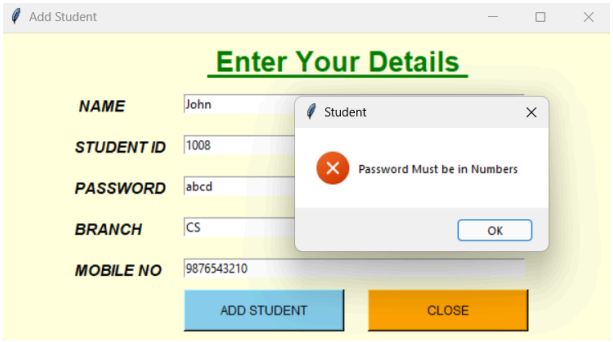
TC-01,TC-02,TC-05, TC-08, TC-09



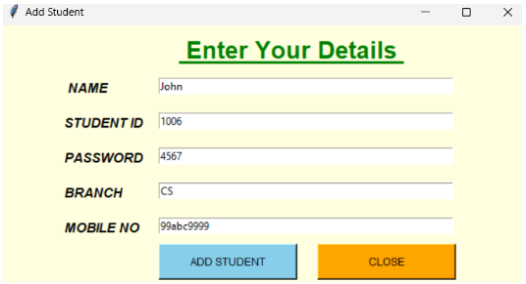
TC-03



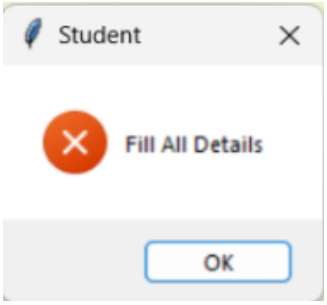
TC-04



TC-06



TC-07



3. Testing Delete Student Feature

i. Comparison with other techniques:

Decision Table Testing (DTT) is ideal for the Delete Student feature as it efficiently handles multiple conditions (admin password, student ID, existence) and their combinations, providing comprehensive test coverage compared to simpler techniques like ECP or BVA.

ii. Why DTT?

DTT is chosen because it helps manage multiple input conditions and actions, ensuring all possible scenarios for the "Delete Student" process are covered in a structured and efficient manner.

iii. Where to apply DTT?

Decision Table Testing (DTT) is used at the system and integration testing levels to validate functional behavior by checking how the system handles various combinations of inputs and conditions. It ensures that different rules and scenarios are correctly implemented across the application.

Testing the Feature

Step 1: Identify Conditions

C1:Is Admin Password correct?
C2: Is Student ID field empty?
C3: Does Student ID exist in DB?

step 2: Identify possible actions

A1: Password is Wrong.
A2: Enter Correct ID

A3:Student Not Found.

A4:Student Deleted Successfully.

step 3: Initial Decision Table

no of columns = 2^3=8

Conditions	R1	R2	R3	R4	R5	R6	R7	R8
is Admin Password correct?	T	T	T	T	F	F	F	F
Is Student ID field empty?	T	T	F	F	T	T	F	F
Does Student ID exist in DB?	T	F	T	F	T	F	T	F—
Action								

Password is Wrong					✔	✔	✔	✔
Enter Correct ID	✔	✔						
Student Not Found				✔				
Student Deleted Successfully			✔					

Step:4 Reduced Table

Conditions	R1-R2	R3	R4	R5-R8
is Admin Password correct?	T	T	T	T
Is Student ID field empty?	T	T	F	F
Does Student ID exist in DB?	T	F	T	F
Action				
Password is Wrong				✔
Enter Correct ID	✔			
Student Not Found		✔		
Student Deleted Successfully			✔	

Project Summary :

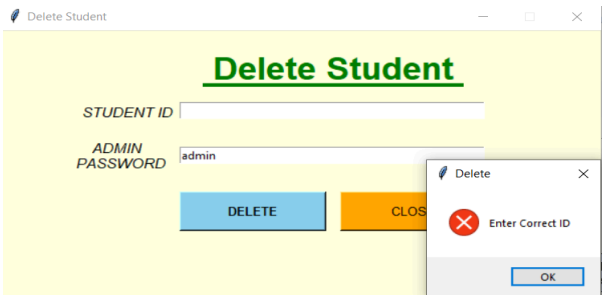
Project Name:	**Delete Student**
Test Suite ID:	DeleteStudent_TESE61
Test Title:	Test the Delete Student
Test Priority:	Medium
Module Name:	Delete Student
Designed By:	Urwa Qadir
Designed Date:	4/30/2025
Executed By:	Urwa Qadir
Executed Date:	4/30/2025
Description of Test:	To check the behavior of the deletestudents() function under different inputs
pre-condition	Login as a Admin

Test Case:

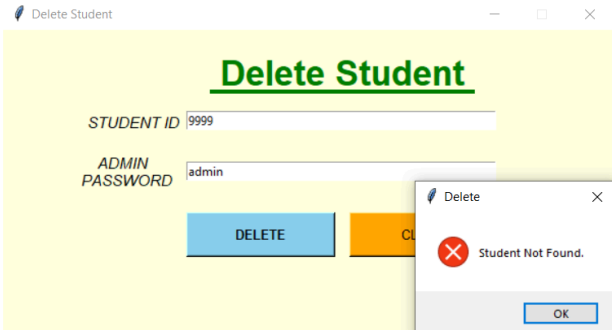
Test Case ID	Test Case Description	Test Data	Expected Result	Actual Result	Status
TC1	Test with correct admin password, empty Student ID field, and valid Student ID in DB	Admin Password: "admin", Student ID: ""	"Enter Correct ID"	"Enter Correct ID"	Pass
TC2	Test with correct admin password, valid Student ID field, and non-existent Student ID in DB	Admin Password: "admin", Student ID: "9999"	"Student Not Found"	"Student Not Found"	Pass
TC3	Test with correct admin password, valid Student ID field, and valid Student ID in DB	Admin Password: "admin", Student ID: "1001"	"Student Deleted Successfully."	"Student Deleted Successfully. "	Pass
TC4	Test with incorrect admin password, valid Student ID field, and valid Student ID in DB	Admin Password: "wrongpass", Student ID: "1001"	"Password is Wrong"	"Password is Wrong"	Pass

Output:

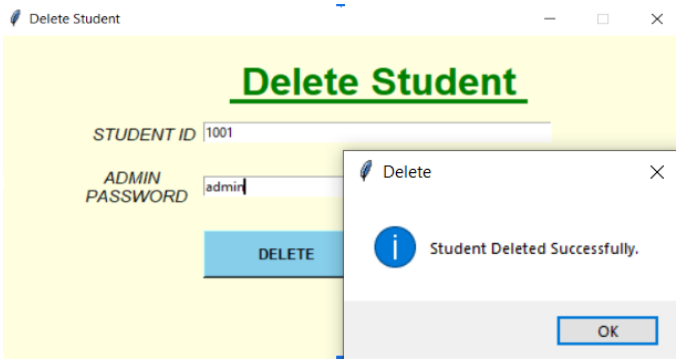
TC_01



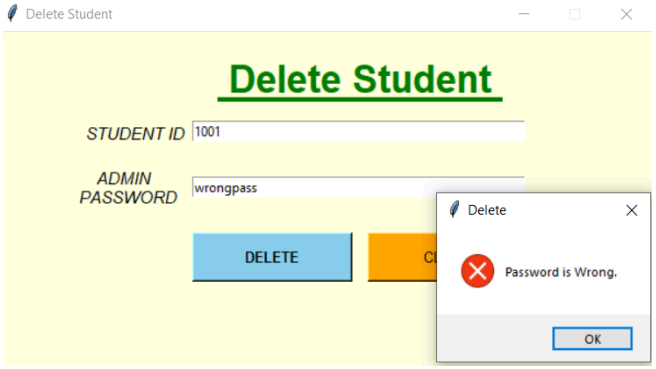
TC_02:



TC_03:



TC_04:



4. Testing Add Book Feature

i. Comparison:

I selected **Equivalence Partitioning** as it's effective for validating grouped input types (e.g., empty vs non-empty, numeric vs non-numeric). BVA focuses on testing edge values, which isn't suitable here since form inputs don't have clear numeric boundaries. Decision Table Testing is useful for complex rules or combinations of conditions, but this feature has simple input checks, making EP more practical and efficient.

ii. Why Equivalence Partitioning technique?

- It efficiently tests different input types with fewer cases.

- Perfect for validating form fields (e.g., empty vs filled, valid vs invalid).
- Easier to apply and understand for user input scenarios.
- Reduces redundancy while still covering major behaviors.

iii. On what level can we apply black box testing and why?

- **System Level:** To test the overall behavior from the user's point of view.
- **Integration Level:** To check interactions between modules (e.g., form and database).
- Useful because it validates functionality without needing to access the internal code, focusing purely on inputs and outputs.

Test Summary:

Project Name	Library Management System
Test Suite ID	AddBook_TESE60
Test Title	Black Box Testing for Add Book Feature
Test Priority	High
Module Name	Add Book
Designed By	Namra Imtiaz
Designed Date	05/05/2025
Executed By	Namra Imtiaz
Executed Date	05/05/2025
Test Technique	Equivalence Partitioning
Test Type	Black Box Testing
Description of Test	To validate the Add Book function with various valid and invalid input partitions
Pre-condition	Admin must be logged in and on Add Book interface

Equivalence Classes Identified:

Input field	**Valid class**	**Invalid Classes**
Subject	Non-Empty String	Empty String
Title	Non-Empty String	Empty String

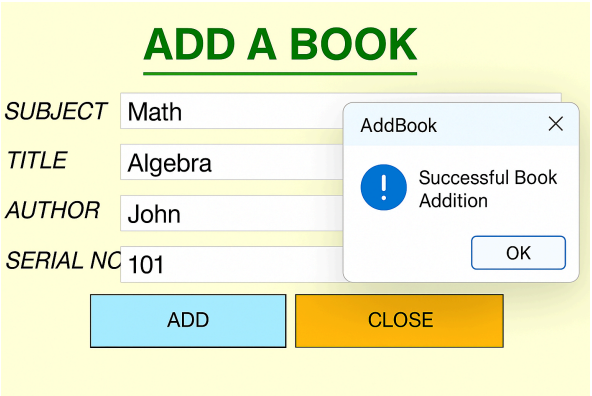
Author	Non-Empty String	Empty String
Serial No	Integer value not in db	non numeric,empty,already exist

Test Cases Using Equivalence Partitioning:

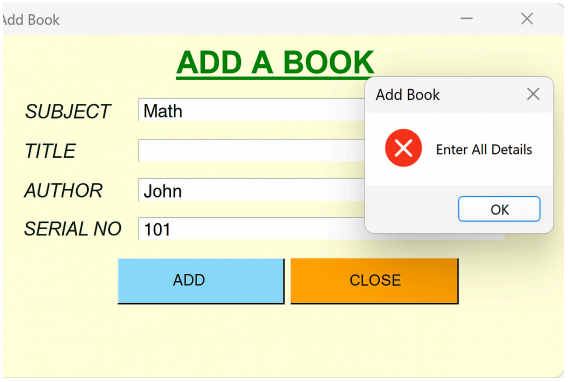
S.No	Test Case ID	Description	Test Data	Expected Result	Status
1	TC_BB_01	All valid inputs	"Math", "Algebra", "John", "101" (new serial)	Book Added Successfully	Pass
2	TC_BB_02	Empty Subject	"", "Algebra", "John", "105"	Show Error: Fill All Details	Pass
3	TC_BB_03	Empty Title	"Math", "", "John", "105"	Show Error: Fill All Details	Pass
4	TC_BB_04	Empty Author	"Math", "Algebra", "", "105"	Show Error: Fill All Details	Pass
5	TC_BB_05	Empty Serial	"Math", "Algebra", "John", ""	Show Error: Fill All Details	Pass
6	TC_BB_06	Non-numeric Serial	"Math", "Algebra", "ABC", "ABC"	Show Error: Serial must be a number	Pass
7	TC_BB_07	Serial Already Exists	"Math", "Algebra", "John", "101" (already in DB)	Show Error: ID Already Taken	Pass

Outputs:

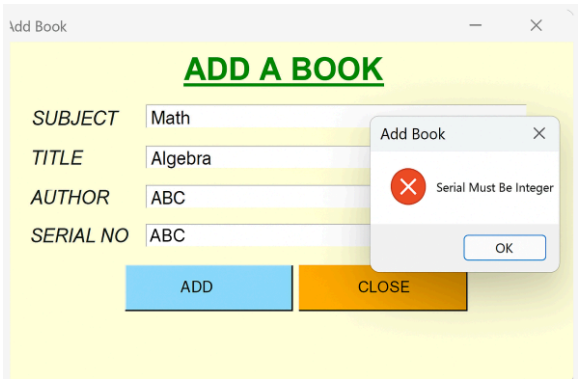
TC_BB_01:



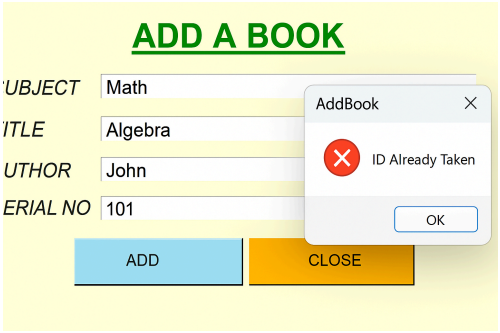
TC_BB_02 to TC_BB_05:



TC_BB_06:



TC_BB_07:



5. Delete Books Feature

i. Comparison of Decision Table Testing with Other Techniques

Decision Table Testing: Ideal for complex scenarios with multiple conditions leading to different actions.

Equivalence Partitioning: Divides input data into valid and invalid partitions, reducing redundant tests.

Boundary Value Analysis: Focuses on testing boundary conditions (maximum and minimum values), but may miss combinations of different conditions.

As per my usecase, the Delete Book feature involves multiple conditions (e.g., valid credentials, book existence, book ID entered), so Decision Table Testing is perfect for managing such complexity. It ensures we test all possible combinations of these conditions.

ii. Why Choose This Technique?

I chose Decision Table Testing because:

It handles scenarios with multiple input combinations, making it perfect for features like Delete Book that depend on various factors like credentials, book existence, and user input.
It ensures thorough coverage by testing all possible paths for each condition.

iii. When and Where Can We Apply Decision Table Testing?

Level: Decision Table Testing is most useful at the system or integration level where business rules or complex logic makes the system outcome. It's best suited for multiple conditions that result in different outcomes.
When to Stop Testing: We stop testing when all possible combinations of conditions have been covered. If all paths pass, we can conclude that the system handles all cases as expected.

Decision Table:

C1: Is the password correct?
C2: Does the book exist in the system?
C3: Is the Book ID provided?

A1: Delete Book.
A2: Show Error not found.

A3: Incorrect Password Error.

A4: Book ID not provided Error.

Initial Decision Table

no of columns = 2^3=8

Conditions	**R1**	**R2**	**R3**	**R4**	**R5**	**R6**	**R7**	**R8**
C1	T	T	T	T	F	F	F	F
C2	T	T	F	F	T	T	F	F

C3	T	F	T	F	T	F	T	F
Action								
A1	✔							
A2			✔	✔				
A3					✔	✔	✔	✔
A4		✔						

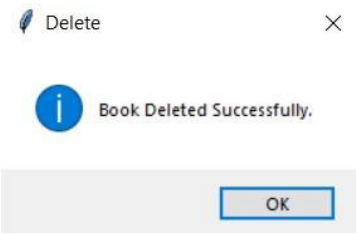
Step:4 Reduced Table

Conditions	**R1**	**R2**	**R3-R4**	**R5-R8**
C1	T	T	T	F
C2	T	T	F	X
C3	T	F	X	X
Action				
A1	✔			
A2			✔	
A3				✔
A4		✔		

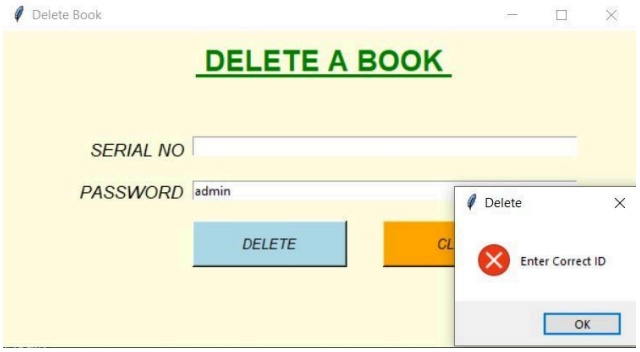
Test Case:

Test Case	Description	Test Data	Expected Output	Actual Output	Status
TC_1	Check if the system allows deletion when all conditions are met.	Username: admin Password: admin Book ID: 101	Book with ID 101 is deleted successfully. Message: "Book Deleted Successfully."	Book Deleted	Pass
TC_2	Check if the system prompts error when no book ID is provided.	Username: adminPassword: adminBook ID: "" (empty)	Error message: "Enter Correct ID."	Error msg	Pass
TC_3	Check if the system shows error when the book does not exist.	Username: admin Password: admin Book ID: 999	Error message: "Book Not Found."	Error MSg	Pass
TC_4	Check if the system rejects deletion when the credentials are wrong.	Username: admin Password: wrong Book ID: 1	Error message: "Password is Incorrect."	Error MSg	Pass

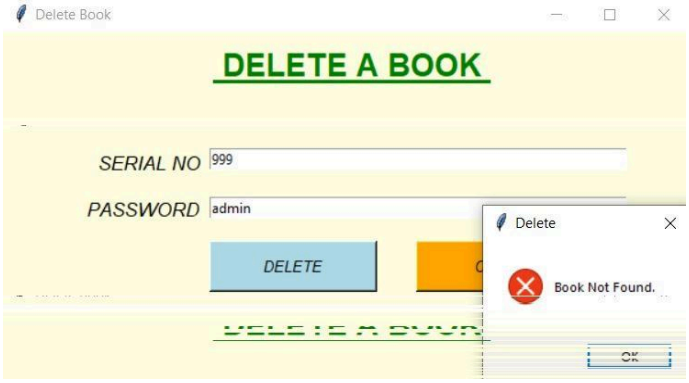
TC_1:



TC_2:



TC_3:



TC_4:

Summary Report

Project Name	Library Management System
Test Suite ID:	BB_LMS-05
Test Title:	Delete Books
Test Priority:	Medium
Module Name:	Delete Books

Designed By:	Jassica Allen
Designed Date:	5/5/2025
Executed By:	Jassica Allen
Executed Date:	5/05/2025
Description of Test:	Testing the delete books feature
PREREQUISITES:	
Pre-Condition:	Logged in as admin
Dependencies:	